Implementación de Listas en Java

Noemi Mata Ruiz
Grupo: GTID244
Estructuras de datos
Fecha: 23/10/2026
Repositorio: [noemi-mata/noemi-mata-listas-java]

Ejercicio 1. Formados en la cafetería

Descripción del problema

El problema trata de una cafetería con dos filas, una de alumnos y otra de trabajadores. Cuando toca atender a alguien se atiende primero a los trabajadores, y a los alumnos solo si no hay ningún trabajador formado. El programa lee una serie de eventos (llega un alumno, llega un trabajador o se atiende a alguien) y va imprimiendo el nombre de las personas que se atienden.

Tipo de lista y por qué la elegí

Usé listas simples, una para cada fila. Una fila solo necesita agregar personas al final y sacar a la primera del inicio, así que no me hace falta recorrerla hacia atrás y una lista doble guardaría un puntero que nunca uso. A la lista le agregué una referencia al último nodo para no tener que recorrerla completa cada vez que llega alguien, porque pueden ser hasta 200 000 eventos.

Explicación del algoritmo

Cuando llega alguien creo un nodo y lo enlazo al final de su fila (cola.siguiente = nuevo y luego cola = nuevo). Cuando hay que atender reviso si la fila de trabajadores tiene gente; si la tiene saco su primer nodo, y si no saco el de la fila de alumnos. Para sacarlo muevo la cabeza al siguiente nodo e imprimo el nombre. Si la fila se queda vacía también pongo cola en null para que no quede apuntando a un nodo que ya salió.

Pruebas realizadas

Probé el programa con casos propios y con el veredicto de omegaUp.

Caso 1: Normal (enunciado)

Entrada:

```
7
LLEGA ALUMNO luis
LLEGA TRABAJADOR ana
LLEGA ALUMNO jorge
ATIENDE
ATIENDE
LLEGA TRABAJADOR juan
ATIENDE
```

Salida esperada:

```
ana
```

```
luis
juan
```

```
C:\Users\Noemi\noemi-mata-listas-java\ejercicio1_FormadosEnLaCafeteria
>java Main
7
LLEGA ALUMNO luis
LLEGA TRABAJADOR ana
LLEGA ALUMNO jorge
ATIENDE
ATIENDE
LLEGA TRABAJADOR juan
ATIENDE
ana
luis
juan

C:\Users\Noemi\noemi-mata-listas-java\ejercicio1_FormadosEnLaCafeteria
>
```

Caso 2: Límite (un solo elemento)

Entrada:

```
2
LLEGA ALUMNO pedro
ATIENDE
```

Salida esperada:

```
pedro
```

```
C:\Users\Noemi\noemi-mata-listas-java\ejercicio1_FormadosEnLaCafeteria
>java Main
2
LLEGA ALUMNO pedro
ATIENDE
pedro

C:\Users\Noemi\noemi-mata-listas-java\ejercicio1_FormadosEnLaCafeteria
```

Caso 3: Prioridad y orden

Entrada:

```
8
LLEGA ALUMNO maria
LLEGA ALUMNO carlos
LLEGA TRABAJADOR sofia
LLEGA TRABAJADOR diego
ATIENDE
ATIENDE
ATIENDE
ATIENDE
```

Salida esperada:

```
sofia
diego
```

```
maria
carlos
```

```
C:\Users\Noemi\noemi-mata-listas-java\ejercicio1_FormadosEnLaCafeteria
>java Main
8
LLEGA ALUMNO maria
LLEGA ALUMNO carlos
LLEGA TRABAJADOR sofia
LLEGA TRABAJADOR diego
ATIENDE
ATIENDE
ATIENDE
ATIENDE
sofia
diego
maria
carlos
```

Caso 4: Llegan trabajadores después

Entrada:

```
6
LLEGA ALUMNO rosa
ATIENDE
LLEGA ALUMNO tomas
LLEGA TRABAJADOR elena
ATIENDE
ATIENDE
```

Salida esperada:

```
rosa
elena
tomas
```

```
C:\Users\Noemi\noemi-mata-listas-java\ejercicio1_FormadosEnLaCafeteria
>java Main
6
LLEGA ALUMNO rosa
ATIENDE
LLEGA ALUMNO tomas
LLEGA TRABAJADOR elena
ATIENDE
ATIENDE
rosa
elena
tomas
```

Caso 5: Grande (N = 200000)

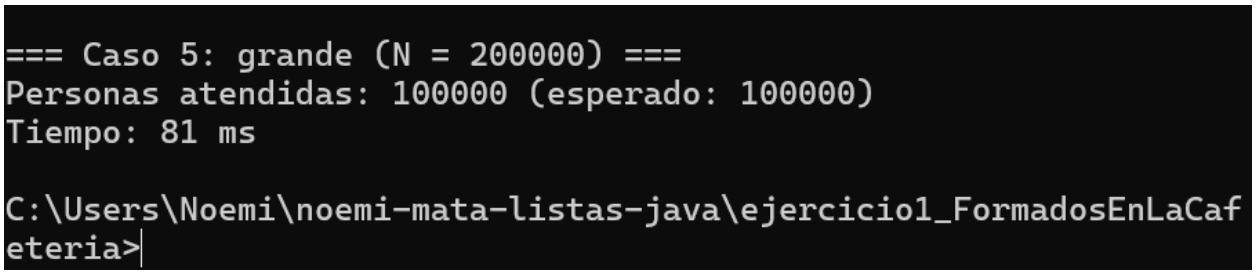
Entrada:

```
N = 200000
100000 llegadas (alternando alumno y trabajador)
```

```
seguidas de 100000 eventos ATIENDE
```

Salida esperada:

```
100000 personas atendidas
(se muestra la cantidad y el tiempo)
```



Puntaje de omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo
2026-10-23 00:54	java	100.00%	Terminada	Correcta	19.77 MB	0.53 s

Enlace al código

[noemi-mata-listas-java/ejercicio1_FormadosEnLaCafeteria at main · noemi-mata/noemi-mata-listas-java]

Conclusiones

La dificultad principal fue el tiempo: agregar al final recorriendo toda la lista habría sido O(n) por llegada y habría excedido el límite con 200 000 eventos; lo resolví guardando una referencia al último nodo. Otra dificultad fue olvidar limpiar la cola cuando la lista se vacía, lo que dejaba un puntero a un nodo ya eliminado; lo corregí asignando cola = null en ese caso.

Ejercicio 3. Comprando agua

Descripción del problema

En una tienda hay N marcas de agua y todas tienen un precio distinto. Como hay que guardar dinero para el camión de regreso, se quiere comprar la más barata. El programa lee la cantidad de marcas y sus precios e imprime el menor.

Tipo de lista y por qué la elegí

Usé una lista simple. Los precios se guardan en el orden en que llegan y para encontrar el menor basta recorrer la lista una vez de principio a fin, así que no necesito el puntero anterior de una lista doble. También guardé una referencia al último nodo para agregar al final sin recorrer la lista.

Explicación del algoritmo

Leo N y por cada precio creo un nodo y lo enlazo al final. Para buscar el menor empiezo suponiendo que el precio de la cabeza es el menor y recorro el resto de la lista con actual = actual.siguiente; cada vez que encuentro un precio más bajo lo guardo como nuevo mínimo. Al terminar imprimo el mínimo.

Pruebas realizadas

Probé el programa con casos propios (uno normal, casos límite y uno grande) y con el veredicto de omegaUp.

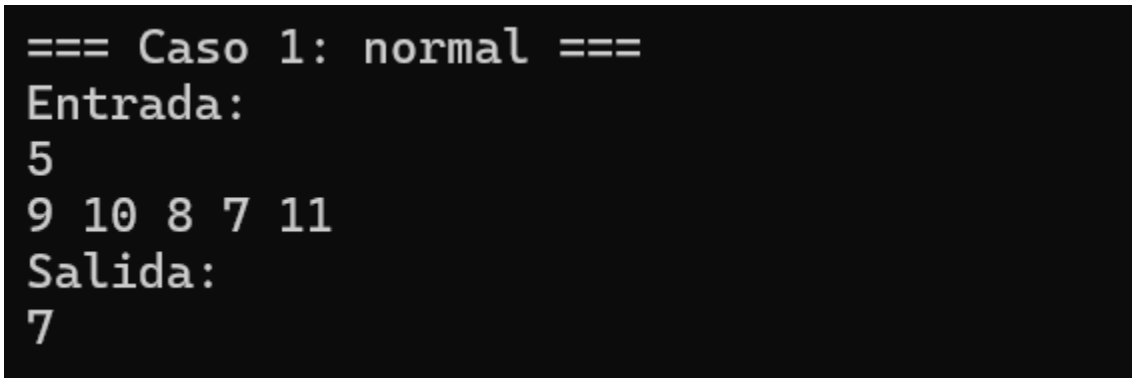
Caso 1: Normal (enunciado)

Entrada:

```
5
9 10 8 7 11
```

Salida esperada:

```
7
```



Caso 2: Normal (enunciado)

Entrada:

```
3
```

```
20 12 16
```

Salida esperada:

```
12
```

```
=== Caso 2: normal ===
Entrada:
3
20 12 16
Salida:
12
```

Caso 3: Límite (una sola marca)

Entrada:

```
1
500
```

Salida esperada:

```
500
```

```
=== Caso 3: limite (una sola marca) ===
Entrada:
1
500
Salida:
500
```

Caso 4: Límite (menor al inicio)

Entrada:

```
4
1 50 99999 300
```

Salida esperada:

```
=== Caso 4: limite (menor al inicio) ===
Entrada:
4
1 50 99999 300
Salida:
1
```

```
1
```

Caso 5: Límite (menor al final)

Entrada:

```
4
900 800 700 2
```

Salida esperada:

```
2
```

```
=== Caso 5: limite (menor al final) ===
Entrada:
4
900 800 700 2
Salida:
2
```

Caso 6: Grande (N = 10000)

Entrada:

```
10000
precios 99999, 99998, … hasta 90000 (decreciente)
```

Salida esperada:

```
90000
```

```
=== Caso 6: grande (N = 10000) ===
Menor precio: 90000 (esperado: 90000)
Tiempo: 0 ms
```

Veredicto de omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-23 01:08	java	100.00%	Terminada	Correcta	0.00 MB	0.65 s	

Enlace al código

[noemi-mata-listas-java/ejercicio3_ComprandoAgua at main · noemi-mata/noemi-mata-listas-java]

Conclusiones

La dificultad principal fue la lectura: los precios pueden venir en una sola línea o en varias, así que leer línea por línea habría fallado; lo resolví leyendo número por número con StringTokenizer. Otra dificultad fue inicializar el mínimo: usar un valor fijo como 0 daba un resultado incorrecto, y lo resolví partiendo del precio del primer nodo.

Ejercicio 5. Supermercado

Descripción del problema

Un supermercado registra N productos con su nombre y su precio. El programa imprime el ticket con los productos ordenados del más caro al más barato, numerados desde 1 con el formato número | nombre | precio, y al final el total de la compra.

Tipo de lista y por qué la elegí

Usé una lista simple que mantengo ordenada desde el principio. Cada producto se inserta directo en su lugar, por lo que no necesito ordenar después ni usar otra estructura. Siempre se recorre hacia adelante, así que no hace falta una lista doble.

Explicación del algoritmo

Cada producto que leo se inserta con insertarOrdenado. Si la lista está vacía o su precio es mayor o igual que el de la cabeza, el nodo nuevo pasa a ser la cabeza. Si no, avanzo con actual = actual.siguiente mientras el siguiente nodo cueste más que el nuevo y ahí lo enlazo (nuevo.siguiente = actual.siguiente y luego actual.siguiente = nuevo). Cuando hay precios iguales el producto que llegó después queda antes, porque así sale en el ejemplo del problema (Colgate antes que fud). Para el ticket recorro la lista con un contador para la posición y voy sumando el total al leer cada precio. Los precios se imprimen sin ceros de más, por ejemplo 23.60 sale como 23.6.

Pruebas realizadas

Probé el programa con casos propios (uno normal, casos límite y uno grande) y con el veredicto de omegaUp.

Caso 1: Normal (enunciado)

Entrada:

```
8
axion 23.60
fud 23.00
LaLa 23.01
Petalo 129.5
2Kg_manzanas 58.6
BimboG 30.5
MayonesaM 50.5
Colgate 23
```

Salida esperada:

```
1 | Petalo | 129.5
2 | 2Kg_manzanas | 58.6
3 | MayonesaM | 50.5
4 | BimboG | 30.5
5 | axion | 23.6
6 | LaLa | 23.01
7 | Colgate | 23
8 | fud | 23
```

```
361.71
```

```
=== Caso 1: normal (enunciado) ===
Entrada:
8
axion 23.60
fud 23.00
LaLa 23.01
Petalo 129.5
2Kg_manzanas 58.6
BimboG 30.5
MayonesaM 50.5
Colgate 23
Salida:
1 | Petalo | 129.5
2 | 2Kg_manzanas | 58.6
3 | MayonesaM | 50.5
4 | BimboG | 30.5
5 | axion | 23.6
6 | LaLa | 23.01
7 | fud | 23
8 | Colgate | 23

361.71
```

Caso 2: Límite (un solo producto)

Entrada:

```
1
leche 25.5
```

Salida esperada:

```
1 | leche | 25.5

25.5
```

```
=== Caso 2: limite (un solo producto) ===
Entrada:
1
leche 25.5
Salida:
1 | leche | 25.5

25.5
```

Caso 3: Precios iguales (el que llega después va antes)

Entrada:

```
3
pan 10
atun 10
sal 10
```

Salida esperada:

```
1 | sal | 10
2 | atun | 10
```

```
3 | pan | 10

30
```

```
=== Caso 3: precios iguales ===
Entrada:
3
pan 10
atun 10
sal 10
Salida:
1 | pan | 10
2 | atun | 10
3 | sal | 10

30
```

Caso 4: Entrada ascendente

Entrada:

```
3
a 1.5
b 2.25
c 3
```

Salida esperada:

```
1 | c | 3
2 | b | 2.25
3 | a | 1.5

6.75
```

```
=== Caso 4: entrada ascendente ===
Entrada:
3
a 1.5
b 2.25
c 3
Salida:
1 | c | 3
2 | b | 2.25
3 | a | 1.5

6.75
```

Caso 5: Suma con decimales

Entrada:

```
2
```

```
cafe 99.99
te 0.01
```

Salida esperada:

```
1 | cafe | 99.99
2 | te | 0.01

100
```

```
=== Caso 5: suma con decimales ===
Entrada:
2
cafe 99.99
te 0.01
Salida:
1 | cafe | 99.99
2 | te | 0.01

100
```

Caso 6: Grande (100 productos)

Entrada:

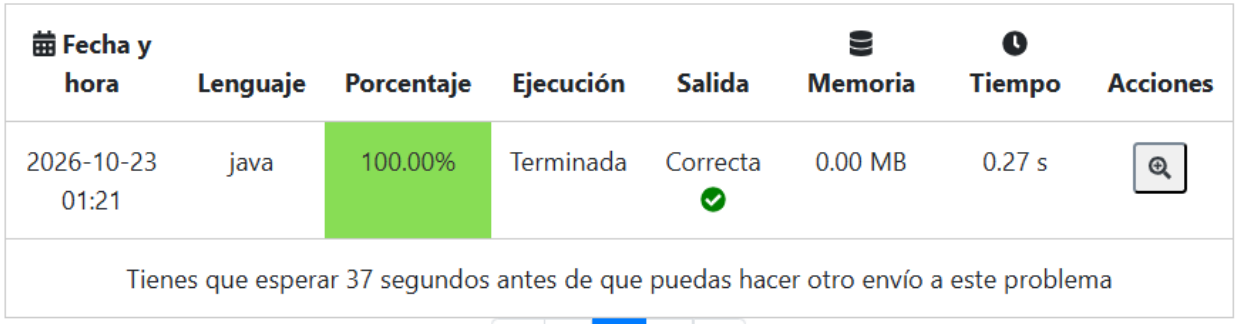
```
100
producto1 2.5, producto2 5, … producto100 250
```

Salida esperada:

```
Primera línea: 1 | producto100 | 250
Última línea: 100 | producto1 | 2.5
Total: 12625
```

```
=== Caso 6: grande (N = 100) ===
Primera línea: 1 | producto100 | 250 (esperado: 1 | producto100 |
250)
Última línea:  100 | producto1 | 2.5 (esperado: 100 | producto1 |
2.5)
Total: 12625 (esperado: 12625)
```

Veredicto de omegaUp



Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-23 01:21	java	100.00%	Terminada	Correcta	0.00 MB	0.27 s	

Tienes que esperar 37 segundos antes de que puedas hacer otro envío a este problema

Enlace al código

[noemi-mata-listas-java/ejercicio5_Supermercado at main · noemi-mata/noemi-mata-listas-java]

Conclusiones

La dificultad principal fue el formato de los precios: al imprimir un double en Java sale 23.0 en lugar de 23, y el enunciado pide 23; lo resolví con una función formatear que quita los ceros sobrantes. Otra dificultad fue el orden de los productos con el mismo precio: al principio dejaba primero al que llegó antes y no coincidía con el ejemplo del problema (Colgate debe salir antes que fud); lo resolví cambiando la comparación para que el nuevo producto se coloque antes de los que cuestan lo mismo.

Ejercicio 7. Cuadro de Honor

Descripción del problema

El programa recibe N alumnos con su nombre y su promedio. Al cuadro de honor solo entran quienes tienen 9.5 o más, y hay que imprimir sus nombres en orden alfabético, uno por línea.

Tipo de lista y por qué la elegí

Usé una lista simple que mantengo ordenada alfabéticamente. Solo guardo a los alumnos que sí entran al cuadro y cada uno se inserta en su lugar, así que al final solo recorro la lista de principio a fin para imprimir y no necesito una lista doble.

Explicación del algoritmo

Leo N y, por cada alumno, una línea con el nombre y otra con el promedio; leo líneas completas porque los nombres tienen espacios. Si el promedio es mayor o igual a 9.5 inserto el nombre en la lista. Si la lista está vacía o el nombre va antes que el de la cabeza, el nodo nuevo pasa a ser la cabeza. Si no, avanzo con actual = actual.siguiente mientras el siguiente nombre sea menor o igual que el nuevo y lo enlazo ahí (nuevo.siguiente = actual.siguiente y luego actual.siguiente = nuevo). Al final recorro la lista imprimiendo los nombres y, si nadie entró, no se imprime nada.

Pruebas realizadas

Probé el programa con casos propios (uno normal, casos límite y uno grande) y con el veredicto de omegaUp.

Caso 1: Normal (enunciado)

Entrada:

```
5
Pepito Perez
9.6
Julian Cardenas
10.0
Felix Garduza
9.9
Martin Gonzalez
9.2
Jose Soto
8.5
```

Salida esperada:

```
Felix Garduza
Julian Cardenas
Pepito Perez
```

```
=== Caso 1: normal (enunciado) ===
Entrada:
5
Pepito Perez
9.6
Julian Cardenas
10.0
Felix Garduza
9.9
Martin Gonzalez
9.2
Jose Soto
8.5
Salida:
Felix Garduza
Julian Cardenas
Pepito Perez
```

Caso 2: Límite (nadie entra)

Entrada:

```
2
Ana Lopez
9.4
Luis Mora
9.49
```

Salida esperada:

```
(no se imprime nada)
```

```
=== Caso 2: limite (nadie entra) ===
Entrada:
2
Ana Lopez
9.4
Luis Mora
9.49
Salida:
(vacía: nadie alcanzó el cuadro de honor)
```

Caso 3: Límite (9.5 exacto sí entra)

Entrada:

```
2
Zoe Ruiz
9.5
Ana Gil
9.50
```

Salida esperada:

```
Ana Gil
Zoe Ruiz
```

```
=== Caso 3: limite (9.5 exacto) ===
Entrada:
2
Zoe Ruiz
9.5
Ana Gil
9.50
Salida:
Ana Gil
Zoe Ruiz
```

Caso 4: Límite (un solo alumno)

Entrada:

```
1
Carlos Vega
10.0
```

Salida esperada:

```
Carlos Vega
```

```
=== Caso 4: limite (un solo alumno) ===
Entrada:
1
Carlos Vega
10.0
Salida:
Carlos Vega
```

Caso 5: Orden alfabético inverso

Entrada:

```
3
Zacarias Luna
10.0
Maria Diaz
9.8
Beto Cruz
9.6
```

Salida esperada:

```
Beto Cruz
Maria Diaz
Zacarias Luna
```

```
=== Caso 5: orden inverso ===
Entrada:
3
Zacarias Luna
10.0
Maria Diaz
9.8
Beto Cruz
9.6
Salida:
Beto Cruz
Maria Diaz
Zacarias Luna
```

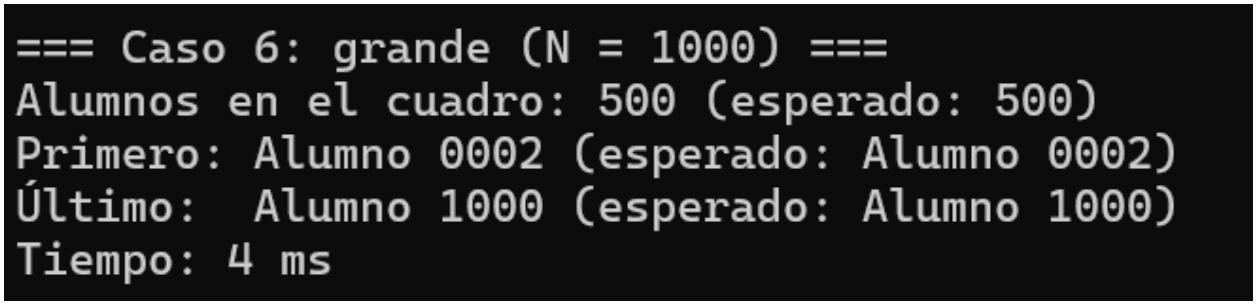
Caso 6: Grande (N = 1000)

Entrada:

```
1000
Alumno 1000 con 10.0, Alumno 999 con 9.0, …
(la mitad entra al cuadro)
```

Salida esperada:

```
500 alumnos en el cuadro
Primero: Alumno 0002
Último: Alumno 1000
```



Veredicto de omegaUp

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-23 01:34	java	100.00%	Terminada	Correcta	0.00 MB	0.70 s	

Tienes que esperar 42 segundos antes de que puedas hacer otro envío a este problema

Enlace al código

[noemi-mata-listas-java/ejercicio7_CuadroDeHonor at main · noemi-mata/noemi-mata-listas-java]

Conclusiones

La dificultad principal fue la lectura: al principio leía palabra por palabra y el nombre Pepito Perez se partía en dos; lo resolví leyendo línea por línea. Otra dificultad fue el límite del promedio: había que usar mayor o igual (>=), porque quien tiene exactamente 9.5 sí entra al cuadro.